Arten

Edge Object Detection, Counting & Attributes

Author: Sujal M H · Solo participant

Namma AI Hackathon 2026 · Problem Statement 2 — AI for Safer Communities

---

Abstract

Arten is an offline edge-AI proof of concept that detects, counts and attributes pedestrians and vehicles in a short street clip or image set. A compact YOLO11n INT8 ExecuTorch model runs on a Raspberry Pi 5 CPU. The pipeline writes per-frame counts, on-device direction, dwell and relative-speed attributes, optional coarse vehicle colour, and annotated frames. On the committed clip it processes 150 frames, peaks at 9 people in one frame and tracks 18 people plus two background vehicles. Arm's model card reports the Pi 5 reference: **134.10 ms p50 / 7.46 FPS**, **59.17 MB** peak memory and **38.90% mAP@0.5:0.95**.

1. Problem and application

Road traffic injuries are the leading cause of death for people aged 5–29 worldwide (WHO). India recorded **1,68,491 road deaths in 2022**; pedestrians were **19.5%** of the fatalities and two-wheeler riders **44.5%** (MoRTH).

A school, ward office or resident group does not need another surveillance platform. It needs a cheap, repeatable answer to one question: *how many people cross here, when, and alongside how much traffic?* That answer is usually locked behind expensive cameras, proprietary dashboards or continuous cloud connectivity.

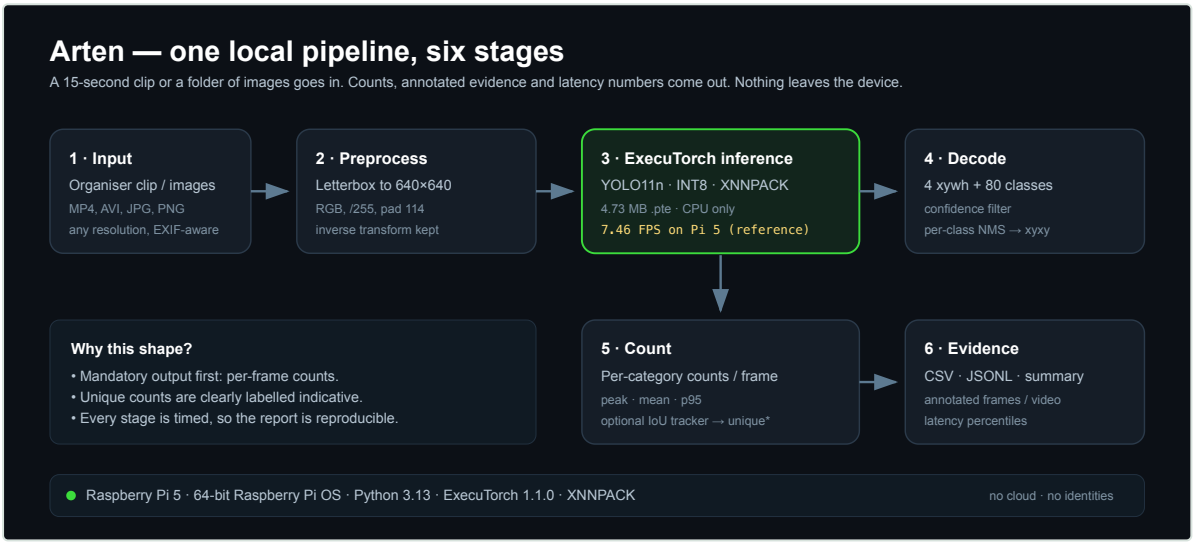
Application. A school records three short clips at the same gate: morning drop-off, afternoon dismissal and a quiet hour. It runs them locally on a Raspberry Pi. The output is a count table, movement attributes and a few annotated frames. A high pedestrian/vehicle overlap at dismissal supports an on-site review for a crossing guard, a zebra crossing or a no-parking zone. The clip supplies evidence for that review; it does not estimate near-misses or crash risk.

2. Approach

The design makes four choices:

1. **Use an edge artifact.** Arm's pre-optimized YOLO11n INT8 `.pte` is small enough for a Pi and verified on the target.
2. **Make counts the first-class output.** Per-frame counts come directly from detections, independent of tracking quality.
3. **Add attributes on the same tracker.** Direction, dwell and relative speed reuse the existing tracker. Optional colour is classical HSV. No second model.
4. **Ship evidence.** Every run writes counts, detections, attribute rows, latency percentiles and annotated frames.

Property	Value
Model	YOLO11n, 80-class COCO detector
Quantization	INT8 static PTQ; stem conv and detection head FP32
Runtime	ExecuTorch 1.1.0, XNNPACK + KleidiAI, CPU only
Pi 5 reference	**134.10 ms p50 · 7.46 FPS · 59.17 MB peak**
COCO accuracy	**38.90% mAP@0.5:0.95 · 54.66% mAP@0.5**
Model size	**4.73 MB**



Pipeline

The pipeline reads video with OpenCV or images with PIL and EXIF transpose, letterboxes to 640×640 with the 114-grey pad, runs the ExecuTorch `forward` method, decodes the 4 xywh + 80 class outputs, applies per-class NumPy NMS and maps boxes back to the original image. It then counts each category, updates the tracker, attaches attributes and writes the evidence bundle.

Attribute	Definition	Limit
Direction	Sign of the smoothed bbox velocity: left, right, up, down, stationary	Image plane, not a compass
Relative speed	Velocity magnitude in pixels/second	Uncalibrated; compare within one clip
Dwell	Time a confirmed track stays visible	Visibility duration, not queue time
Vehicle colour	HSV histogram over the central crop	Heuristic; sensitive to light, glass and shadows
Zone co-presence	Counts inside a manual polygon	Descriptive, not near-miss or crash risk

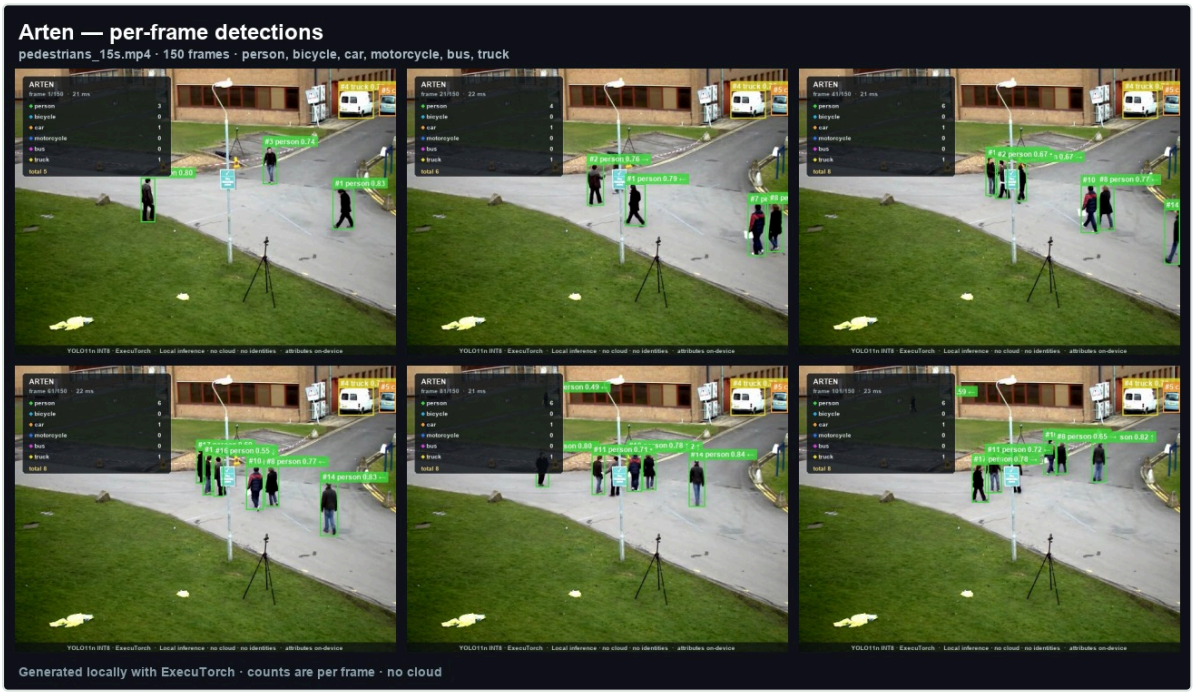
The layer is deliberately non-learned. It adds explainable movement context without face, plate, age, gender or identity attributes.

3. Results

3.1 Pedestrian clip

Category	Peak/frame	Mean/frame	p95	Unique (indicative)
person	9	5.50	8	18
bicycle	0	0.00	0	0
car	1	1.00	1	1
motorcycle	0	0.00	0	0
bus	0	0.00	0	0
truck	1	1.00	1	1

The 18 confirmed person tracks split **9 left, 4 right, 2 stationary, 2 down, 1 up**, with **mean dwell 4.52 s** and **mean relative speed 35.63 px/s**. The car and truck are a blue car and a white van in the background; both are real but stationary. The unique count is indicative: tracker IDs can switch when people cross or leave frame. The per-frame counts are the stable output.



Pedestrian contact sheet

3.2 Dense Bengaluru traffic — stress test



Dense traffic

Detected: **17 cars, 1 bus, 1 truck, 2 people**. The colour heuristic labels the 17 cars as **9 white, 4 gray/silver, 2 blue, 1 red, 1 orange/brown**; the bus is white and the truck orange/brown. The labels are visibly imperfect — the red truck is labelled orange/brown — so they are a demo of derived attributes, not a reliable colour measurement. The scene also exposes the model's limits: **0 motorcycles** detected despite many visible riders, and no auto-rickshaw class.

A close-range street image finds **8 people, 2 cars, 1 motorcycle**; the auto-rickshaw in the centre is missed. Large, close objects are detected confidently; small or non-COCO objects are not.

3.3 Latency

Development host: Apple Silicon macOS, ExecuTorch 1.5.1, torch 2.15.0, CPU. **This is not a Raspberry Pi measurement.**

Stage	Mean ms	p50 ms	p95 ms	FPS from mean
Model inference	21.79	21.70	22.85	45.90
Preprocess	0.70	0.66	0.90	1431.88
Decode + NMS	0.94	0.88	1.02	1061.10
Frame read	0.21	0.17	0.24	4818.64
Annotate/save	3.43	3.25	4.20	291.60
Pipeline total	**27.40**	**27.14**	**28.48**	**36.49**

A standalone benchmark on a real frame measured **20.34 ms mean / 49.16 FPS** with the newer runtime stack. The exact Pi stack (ExecuTorch 1.1.0 + torch 2.10.0) measured **23.83 ms mean / 22.52 ms p50** on the same host. Those are dev-host numbers. Arm's Pi 5 reference is **134.10 ms p50 / 7.46 FPS**. Annotated-video encoding is an output step and is excluded from the pipeline timer.

3.4 Mandatory deliverables

Requirement	Artifact
Process a short clip or image set on ExecuTorch + Pi	Committed sample clip; `arten run`; Pi setup script
Detect a category and show per-frame counts	`counts.csv`; `summary.txt`; annotated HUD
Category + counts + two sample outputs	Pedestrian contact sheet; annotated traffic image
Report inference time, model and throughput	`summary.json`; `benchmark_dev.json`; §3.3
Explain the issue and one application in the short pitch	`docs/pitch-2min.md`; §1

4. Limitations and responsible AI

- **COCO is not India.** Auto-rickshaws, hand carts and many local vehicle types are not COCO classes. The dense traffic image finds 17 cars but no motorcycles.
- **No ground-truth mAP on organiser data.** The report cites COCO reference accuracy and qualitative counts, not a fabricated accuracy on the clip.
- **Attributes are derived, not learned.** Direction is image-plane, speed is uncalibrated pixels/second, colour is a coarse HSV heuristic, and tracker IDs can switch. They support within-clip comparison, not enforcement.
- **Aggregate, not identity.** The system extracts no faces, plates, age, gender or identity fields. Annotated frames still contain the original pixels, so treat them as sensitive evidence and blur them before publication.
- **Licence.** YOLO11n and the Arm `.pte` are AGPL-3.0; this repository is AGPL-3.0 and includes the model notice.

5. Reproduce

```
bash scripts/setup_pi.sh
source .venv/bin/activate
arten doctor
arten run -i data/samples/pedestrians_15s.mp4 \
  -o results/pi_run --save-video --estimate-vehicle-colors
arten benchmark --input data/samples/bengaluru_traffic.jpg --runs 50
```

Model SHA-256: `154c54b52389219ebd982e1003227c1abbc3f29a99bdad241147b684d3041b96`.

Sample media, model card and licences are documented in `data/README.md` and `models/README.md`.
Source: https://github.com/sujalmh/arten-edge-ai.

References

1. WHO. *Road traffic injuries* fact sheet, 2026. https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries
2. MoRTH, Government of India. *Road Accidents in India 2022*, 2023. https://morth.gov.in/backend/documents/uploaded/1755600426_RA_2022_30_Oct.pdf
3. Arm. *YOLO11n INT8 ExecuTorch for Raspberry Pi 5*. https://huggingface.co/Arm/yolo11n-int8-xnnpack-executorch-raspberrypi5
4. Ultralytics. *ExecuTorch integration*. https://docs.ultralytics.com/integrations/executorch
5. PyTorch. *ExecuTorch documentation*. https://docs.pytorch.org/executorch/
6. Namma AI Hackathon. https://www.coe-iot.com/namma-ai-hackathon/